

DATA STRUCTURES · COURSE PROJECT

safetrail

A hand-written geofencing engine for tourist safety in low-connectivity terrain.

SIH 2025 · Problem statement SIH25002 · Ministry of
Development of North Eastern Region

github.com/adivishall/safetrail ·
adivishall.github.io/safetrail

Know when a tourist walks into danger – in real time.

Remote Northeast India. Difficult terrain, patchy mobile coverage, tourists who don't know the local hazards:

- ▶ landslide slopes and quarry edges
- ▶ restricted border zones
- ▶ deep-water lakeshores and seasonal river crossings

We took the problem – not the expected solution.

Every existing implementation hands the hard part to a database. We wrote that engine ourselves.

THE HARD PART	THEY USE	WE DID
"Is this point inside this zone?"	PostGIS ST_Contains	hand-wrote the geometry
"Which zones are near here?"	PostGIS GiST tree	hand-wrote the spatial index
Tamper-proof records	Ethereum contract	hand-wrote a Merkle log

This makes it a data-structures project instead of a web app
– and fixes problems delegating to a cloud DB can't.

03 · WHY IT MATTERS – THE ELEVEN GAPS (8 BUILT)

- 1 GPS is not a point → answer **Inside / Outside / Uncertain**
- 2 Alerts fire too late → **predict** the crossing ("ETA 4 min")
- 3 Zones are static → they turn on/off, and we can **rewind time**
- 4 Tourists tracked alone → model **groups**, detect stragglers
- 5 40 alerts for one landslide → **correlate** into one incident
- 7 Continuous GPS kills battery → **sample by proximity** to danger
- 8 Drift fires fences constantly → **hysteresis**, -91% false alerts
- 9 Ethereum for tamper-proofing → **Merkle log**, offline, no chain

Research + citations in GAP_ANALYSIS.md. Three more are designed, not yet built – marked honestly.

Every second, for each tourist:

- ▶ **Narrow down** — which zones are even *near* them? A tree answers without checking all of them (a book's index vs reading every page).
- ▶ **Check precisely** — are they actually inside? Real polygon geometry.
- ▶ **Report only changes** — "entered" / "exited", never "still inside" forever. That's a usable alert list vs a scrolling wall.

It all lives in one function, `Evaluator::evaluate()`:

1. usability gate	discard noisy fixes	$O(1)$	
2. spatial prune	100k zones $\rightarrow$ ~2 candidates	$O(\log n + k)$	$\leftarrow$ the point
3. temporal filter	which are in force now?	$O(1)$	
4. exact geometry	Inside / Outside / Uncertain	$O(k \cdot V)$	
5. hysteresis	real crossing, or GPS jitter?	$O(1)$	
6. transition diff	did state CHANGE? $\rightarrow$ Event	$O(k)$	
7. prediction	heading toward a zone? $\rightarrow$ ETA	$O(k \cdot V)$	

Step 2 is the whole performance story. Without the index:

$200 \times 100,000 \times 40 = 800$ million ops/tick. With it:

~27,000.

All hand-written – no `std::map/set/priority_queue`, no Boost, no PostGIS.

STRUCTURE	ROLE	COMPLEXITY
Quadtree	primary spatial index	$O(\log n + k)$ avg
R-tree	second index, the comparison	$O(\log n + k)$ avg
Brute force	correctness oracle + baseline	$O(n)$
AVL interval tree	zone validity in time	$O(\log n + k)$ guaranteed
Circular buffer	GPS history per tourist	$O(1)$
Persistent quadtree	time-travel queries	$O(\text{depth})/\text{change}$
Rollback union-find	group cohesion	$O(\log n)$ find
Merkle tree (RFC 6962)	tamper-evident log	proof $O(\log n)$

Plus SHA-256 from scratch, ray casting, winding number, three-valued containment.

- ▶ **Persistent quadtree** – nodes are immutable; a change copies one path and *shares* the rest by reference count. Whole history for a fraction of the cost: **13× sharing** across 5,000 versions. Answers "what were the rules at 14:32?"
- ▶ **Rollback union-find** – groups split as well as merge, so we must *undo* unions → can't use path compression. Union-by-rank alone: $O(\log n)$ find, $O(1)$ rollback. The trade-off is the analysis.
- ▶ **Merkle evidence log** – every event committed to a root; inclusion proofs verify offline; consistency proofs show the log was only appended to. Ethereum's property, in 300 lines that are ours.

- ▶ **Geography: real.** 38 zones from **OpenStreetMap** via Overpass – actual reservoirs, forests, and landmarks around Shillong (**Wards Lake**, **Sonapani Waterfall Cliff**) at true coordinates.
- ▶ **Tourists: simulated.** No real tracking data exists – and simulation gives **ground truth**: we know where each tourist truly was, so we can *measure* if the engine was right.
- ▶ **Reaches the dashboard without travelling.** Zero network requests. One HTML file, 99.3% data, opens over file:// with the network off.

09 · RESULTS — MEASURED, NOT CLAIMED

Spatial index speedup (100k zones)	32.7× / 32.5×	
Candidate pruning (real run)	181×	
Hysteresis false-alert removal	91.2%	realistic drift
Persistent index sharing (5k versions)	13×	
Index equivalence	0 mismatches	18,000 queries
Ray casting vs winding number	0 disagreements	100k points
Unit tests	233 checks	12 files, all pass

We predicted a $\sim 29,000\times$ speedup. Measurement brought it to **33x** – and explaining *why* is worth more than the big number.

At 100k dense zones, ~ 99 genuinely overlap each query. Those are **correct answers** – no index can return fewer results than exist. The ceiling is output size, exactly as $O(\log n + k)$ predicts.

Stating that honestly is the point. Three real bugs were caught only by the measurements – one by looking at the visualisation.

- ▶ **Worst case:** quadtree/R-tree are $O(n)$ worst case (they partition space, not data). The **AVL interval tree** is the one with a guaranteed $O(\log n)$.
- ▶ **Scope:** 23 modules built, ~13 designed-but-stubbed. Marked ☐/✅ throughout – never a stub claimed as done.
- ▶ **The tourists are simulated.** Real geography, simulated people, on purpose.
- ▶ **The dashboard is a deterministic replay,** not a live server.

This is a strength, not a confession – it's what separates a graded project from a sales pitch.

```
make test      # 233 checks pass – proves it works
make demo      # events stream over real geography
make bench     # the speedup + correctness numbers
make dashboard # open dashboard.html – scrub the timeline
```

- ▶ Dots moving over **real Shillong reservoirs**; zones light up on entry.
- ▶ Toggle **index overlay** – the actual quadtree cells (this made a bug visible and doubled our performance).
- ▶ **Scrub past 00:45** – Wards Lake & Love Jungle activate and lapse. The persistent index answering "what were the rules then?".

THE CLOSE

We built the geofencing engine every competing team imports from PostGIS – the quadtree, the R-tree, the persistent time-travel index, the containment geometry, a tamper-evident Merkle log – by hand, analysed it honestly including where it degrades, measured it against real OpenStreetMap geography, and shipped it as a single self-contained dashboard anyone can open offline.

Thank you. · github.com/adivishall/safetrail